The Journal of Open Source Software

CompBind: A Python package for competitive binding analysis of transmembrane protein interfaces from coarse-grained molecular dynamics trajectories

Takeshi Sato [1]

1 Kyoto Pharmaceutical University

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 09 August 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Transmembrane (TM) proteins often utilize multiple contact interfaces for dimerization and oligomerization, with different regions competing for interaction partners. Understanding which interface dominates under specific conditions is crucial for elucidating membrane protein function and regulation. `CompBind` (Competitive Binding Analysis) is a Python package specifically designed to analyze and compare multiple contact interfaces in TM proteins from coarse-grained (CG) molecular dynamics simulations. By leveraging CG representations (e.g., Martini force field), the package enables analysis of long-timescale simulations (microseconds to milliseconds) necessary to observe TM protein association/dissociation events and interface switching dynamics. The package quantifies the competitive advantage between different TM binding regions (e.g., N-terminal vs C-terminal interfaces), calculates their relative binding free energies, and provides rigorous statistical assessment through Bayesian inference.

Statement of need

Transmembrane proteins represent approximately 30% of all proteins and are crucial drug targets. Many TM proteins form dimers or oligomers through specific helix-helix interactions, often involving multiple potential contact interfaces (**?**; **?**). However, capturing TM protein association and interface competition requires simulations on microsecond to millisecond timescales, which are computationally prohibitive with all-atom models but accessible through coarse-grained approaches (**?**; **?**).

Current CG MD analysis tools lack specialized functionality for:

1. Analyzing multiple TM contact interfaces from CG trajectories
2. Handling the reduced resolution of CG models while maintaining biological relevance
3. Quantitative comparison of competing binding regions specific to CG representations
4. Statistical assessment accounting for the enhanced sampling of CG simulations

`CompBind` addresses these needs by providing an integrated workflow specifically tailored for CG simulations of TM proteins. The package is particularly valuable for studying: - Large-scale TM protein oligomerization accessible only through CG simulations - Competition between different TM interfaces over microsecond timescales - Lipid-mediated effects on TM interface preferences in complex membrane compositions - Systems with multiple TM proteins where all-atom simulations are intractable

Methodology

Coarse-Grained Representation and Analysis

The package is optimized for CG trajectories, particularly those using the Martini force field (**?**; **?**), where: - Each amino acid is represented by 1-4 beads (backbone + sidechain) - TM helices maintain secondary structure through elastic networks or virtual sites - Lipids are represented by 4-12 beads depending on complexity

CG-Specific Contact Criteria

For CG models, contact detection uses appropriate distance cutoffs. Two TM segments are considered in contact if their CG beads are within the cutoff distance (default: 10 Å), accounting for the larger effective radius of CG beads compared to atoms.

TM Interface Definition in CG Models

Users define TM regions based on residue numbers, which are automatically mapped to CG beads. For example, in analyzing EGFR-like TM domains, users can define N-terminal and C-terminal interfaces that may compete for dimerization partners.

Free Energy Calculation from CG Trajectories

The binding free energy calculation accounts for the enhanced sampling in CG simulations:

$$\Delta G^{\text{CG}}_{\text{interface}} = -RT_{\text{eff}} \ln \left(\frac{P_{\text{contact}}}{1 - P_{\text{contact}}} \right)$$

where T_{eff} can be adjusted to account for the accelerated dynamics in CG models (typically 2-4x for Martini) (**?**).

The competitive advantage between two TM interfaces is quantified as:

$$\Delta\Delta G = \Delta G_{\text{N-term}} - \Delta G_{\text{C-term}}$$

where negative values indicate preference for the N-terminal interface.

Bayesian Framework Adapted for CG Data

Accounting for CG Simulation Characteristics

The Bayesian model incorporates features specific to CG simulations:

1. **Enhanced sampling correction**: CG simulations sample conformational space more efficiently
2. **Reduced degrees of freedom**: Fewer particles lead to smoother free energy landscapes
3. **Longer trajectories**: Microsecond simulations provide better statistics

Hierarchical Model

The analysis uses a two-level hierarchy appropriate for CG data:

1. **Overall TM dimerization** (observable at CG resolution):

$$\theta^{\text{CG}}_{\text{dimer}} \sim \text{Beta}(1, 1)$$

$$Y_{\text{dimer}} \sim \text{Binomial}(N^{\text{CG}}_{\text{frames}}, \theta^{\text{CG}}_{\text{dimer}})$$

DRAFT

2. **Interface-specific contacts** (CG bead-level):

$$\theta^{\text{CG}}_{\text{interface}} \sim \text{Beta}(1, 1)$$

$$Y_{\text{interface}} \sim \text{Binomial}(N_{\text{dimer}}, \theta^{\text{CG}}_{\text{interface}})$$

Posterior Inference

The posterior distributions are sampled using the No-U-Turn Sampler (NUTS) (**?**), an adaptive variant of Hamiltonian Monte Carlo. We typically use 2000 samples with 1000 tuning steps across 4 chains, with convergence assessed using the Gelman-Rubin statistic ($\hat{R} < 1.01$).

Quantifying Interface Competition

The competitive advantage between regions is quantified through:

1. **Free energy difference**: $\Delta\Delta G = \Delta G_{\text{region1}} - \Delta G_{\text{region2}}$
2. **Posterior probability of preference**: The fraction of posterior samples where region 1 has lower free energy than region 2:

$$P(\text{region1 preferred}) = P(\Delta G_{\text{region1}} < \Delta G_{\text{region2}})$$

3. **95% Highest Density Interval (HDI)**: Provides credible intervals for all quantities

Time-resolved Analysis for Long CG Trajectories

CG simulations often span microseconds, allowing observation of: - Multiple association/dissociation events - Interface switching on biologically relevant timescales - Lipid-mediated effects on interface preference - Oligomerization beyond dimers

Implementation

Core Dependencies and CG Compatibility

`CompBind` leverages: - MDAnalysis (Gowers et al., 2016; Michaud-Agrawal et al., 2011) with CG topology support - NumPy (Harris et al., 2020) and SciPy (Virtanen et al., 2020) for numerical computations - PyMC3 (Salvatier et al., 2016) and ArviZ (Kumar et al., 2019) for Bayesian analysis - Matplotlib (Hunter, 2007) for CG-appropriate visualizations

CG-Specific Optimizations

- **Bead mapping**: Automatic conversion between residue and CG bead representations
- **Trajectory handling**: Efficient processing of long CG trajectories (>100,000 frames)
- **Distance calculations**: Optimized for reduced particle numbers in CG systems
- **Membrane boundary conditions**: Proper handling of periodic boundaries in membrane systems

Validation for CG Models

The package validation includes: - Comparison with experimental TM dimerization data - Backmapping to all-atom representations for selected frames - Cross-validation with different CG force fields (Martini 2/3, SIRAH) - Temperature scaling factors for free energy calculations - Convergence diagnostics (Gelman-Rubin $\hat{R} < 1.01$, ESS > 400)

DRAFT

Example Application: CG TM Protein Analysis

“‘python from config import AnalysisConfig from run_interactive import run_full_analysis

Configure for Martini CG simulation

config = AnalysisConfig() config.psf_file = “martini_membrane_system.psf” # Martini topology config.xtc_file = “production_10us.xtc” # 10 microsecond CG trajectory config.contact_cutoff = 10.0 # Appropriate for Martini beads

Define competing TM interfaces (residue numbers mapped to CG beads)

config.add_region(“TM_N_term”, resid_start=645, resid_end=655, description=“N-terminal interface (GxxxG motif)”) config.add_region(“TM_C_term”, resid_start=656, resid_end=671, description=“C-terminal interface”)

Temperature factor for Martini (optional)

config.temperature_factor = 2.5 # Accounts for accelerated dynamics

Run analysis

results = run_full_analysis(config)

Output adapted for CG analysis:

- Interface preferences over microsecond timescales

- Multiple binding/unbinding events statistics

- Lipid-mediated effects on interface selection

Applications

`CompBind` is particularly suited for CG simulations of:

- **Receptor tyrosine kinases**: Competition between different TM dimerization interfaces affecting activation
- **GPCRs**: Alternative TM-TM interfaces in homo- and heterodimerization
- **Ion channels**: Multiple TM contact interfaces in subunit assembly
- **Viral fusion proteins**: Large-scale conformational changes during membrane fusion
- **Membrane protein crowding**: Multiple TM proteins in realistic membrane densities
- **Complex lipid compositions**: Effects of cholesterol, PIP2, cardiolipin on TM interfaces
- **Designed TM peptides**: Optimization of specific interface preferences

Performance

Typical performance for CG trajectories on Apple Silicon (M1 MacBook Pro):

DRAFT

- 10 µs simulation (100,000 frames): ~5 minutes
- 100 µs simulation (1,000,000 frames): ~45 minutes
- Systems with 10-50 TM proteins: scales linearly

The reduced particle count in CG models enables analysis of much larger systems and longer timescales than all-atom simulations.

Limitations and Future Directions

Current limitations:

- Optimized for Martini-style CG models; other CG schemes may require adaptation
- Atomic-level details of interfaces cannot be resolved
- Temperature scaling factors are approximate

Future developments will include:

- Support for polarizable CG models (Martini 3)
- Hybrid CG/all-atom analysis workflows
- Machine learning-based backmapping for detailed interface analysis

Platform Considerations

The package has been developed and tested on Apple Silicon Macs running macOS 14+, leveraging the ARM architecture's efficiency for processing large CG trajectories. The Bayesian analysis module uses PyMC3 v3.11.6 for stable performance on ARM64. Users on Intel-based systems or Linux may need to adapt the installation process.

Acknowledgements

We acknowledge funding contributions from Kyoto Pharmaceutical University Fund for the Promotion of Collaborative Research.

References

Gowers, R. J., Linke, M., Barnoud, J., Reddy, T. J. E., Melo, M. N., Seyler, S. L., Domański, J., Dotson, D. L., Buchoux, S., Kenney, I. M., & Beckstein, O. (2016). MDAnalysis: A python package for the rapid analysis of molecular dynamics simulations. *Proceedings of the 15th Python in Science Conference*, 98–105. https://doi.org/10.25080/Majora-629e541a-00e

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Kumar, R., Carroll, C., Hartikainen, A., & Martin, O. (2019). ArviZ a unified library for exploratory analysis of bayesian models in python. *Journal of Open Source Software*, *4*(33), 1143. https://doi.org/10.21105/joss.01143

Michaud-Agrawal, N., Denning, E. J., Woolf, T. B., & Beckstein, O. (2011). MDAnalysis: A toolkit for the analysis of molecular dynamics simulations. *Journal of Computational Chemistry*, *32*(10), 2319–2327. https://doi.org/10.1002/jcc.21787

Salvatier, J., Wiecki, T. V., & Fonnesbeck, C. (2016). Probabilistic programming in python using PyMC3. *PeerJ Computer Science*, *2*, e55. https://doi.org/10.7717/peerj-cs.55

DRAFT

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

DRAFT